* (B) Search in 2D matrix

① **Matrix:** 2D array → ele. access → arr[row][col] = element

② **Linear search in 2D matrix:** T.C → $O(n \times m)$ or $O(n^2)$
rows, cols; n = rows / cols.

Q.1] Search in row-wise & column-wise sorted matrix:-

→ i.e. matrix in which → every row is sorted
→ every col is sorted.

→ e.g.

	col			
row →	10	20	30	40
	15	25	35	45
	28	29	37	49
	33	34	38	50

→ optimal algo :- staircase search.

① start position: row = 0, col = last col → top-right ele.

② if matrix[row][col] == target → target found.
→ if matrix[row][col] > target → col--; i.e. move left.
→ if matrix[row][col] < target → moredown → row++;

③ stop when row >= n or col < 0

→ T.C = $O(n+m) \approx O(n)$ (rows, cols; rows/cols → larger no. will dominate)

→ S.C = O(1)

Q.2] Search in strictly sorted matrix:-

→ i.e. → each row is sorted, each col is sorted &&
→ first ele. of next row > last ele. of pre. row.

→ e.g.

	cStart	cMid	cEnd
rStart →	1	2	3 4
	5	6	7 8
rEnd →	9	10	11 12

target = 7

→ rows = 4
→ columns = 4

① **Approach 1:** Convert to 1D + B.S.

i) Treat matrix as :- [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12] (mid = 5, ~~columns = 12~~)

ii) Index Mapping :- row = mid / columns
col = mid % columns

→ e.g. i.e. to get ~~no. of~~ rows & cols no. of actual matrix. [current ele. when we found in a target.]

e.g. target = 6 ⇒ row = 5/4 = 1
col = 5 % 4 = 1

i.e. middle ele.

⇓

→ arr[1][1] = target = 6 ⇒ to return row Index & col Index of target. we use mapping.

* T.C ⇒ $O(\log(n \times m))$
(no. of row × no. of cols = no. of elements.)

* S.C ⇒ O(1)

② **Approach 2:** algorithm:-

① Apply B.S on middle col → cMid

② → keep eliminating rows → finally only 2 rows remain.

③ Check middle col. of 2 rows
→ if target not found.
→ search using B.S :-

1. Top-Left — cStart → cMid-1 & rStart
2. Top-Right — cMid+1 → cEnd & rStart
3. Bottom-Left — cStart → cMid-1 & rStart+1
4. Bottom-Right — cMid+1 → cEnd & rStart+1

* T.C ⇒ $O(\log(n) + \log(m))$
(cMid B.S; no. of rows; no. of cols)

* S.C ⇒ O(1) (remaining space B.S on)

while cond → (rStart < (rEnd - 1))
[while eliminating rows]

code:- Approach ②

```
import java.util.Arrays.
public class Main{
    public static void main(String[] args){
        int[][] arr={{1,2,3,4},{5,6,7,8},{9,10,11,12},{13,14,15,
        int target=12;
        int[] emptArrAns={-1,-1};
        if(arr.length==0){
            System.out.println(Arrays.toString(emptArrAns));
            System.exit(0);
        }
        int rows=arr.length;
        int cols=arr[0].length;
        int cMid=cols/2;
        if(rows==1){
            int[] ans=BS(arr,0,0,cols-1,target);
            System.out.println(Arrays.toString(ans));
            System.exit(0);
        }
        int rStart=0;
        int rEnd=cols-1;
        //run till 2rows are remaining. log2(n)→n=no.of rows
        while(rStart<(rEnd-1)){
            int rMid=rStart+(rEnd-rStart)/2;
            if(arr[rMid][cMid]>target){
                rEnd=rMid;
            }
            else{
                rStart=rMid;
            }
        }
        //check whether the target is in the col of 2 rows. log2(m)→as we will now search in cols
        if(arr[rStart][cMid]==target){
            int[] b={rStart,cMid};//if ans. is 1st ele. in middle col.=5
            System.out.println(Arrays.toString(b));
            System.exit(0);
        }
```

e.g.

	1	2	3
rStart →	1 4	5	6 2
rEnd →	3 7	8	9 4

if target=9

→hence changing rStart & rEnd.

rStart = 1 < rEnd = 2-1 → hence cond^n breaks, loop ends here

when only 2 rows are remaining.

```
if (arr[rStart+1][cMid] == target) {
        int[] c = {rStart+1, cMid};
        System.out.println(Arrays.toString(c)); //6-target.
        System.exit(0);
    }
//search in first half
if(arr[rStart][cMid-1] >= target){
        int[] ans = BS(arr, rStart, 0, cMid-1, target);
        System.out.println(Arrays.toString(ans)); ①4]
        System.exit(0);
    }
//search in 2nd half [10 ②
if(arr[rStart][cMid+1] <= target && arr[rStart][col-1] >= target){
    int[] ans = BS(arr, rStart, cMid+1, cols-1, target);
    System.out.println(Arrays.toString(ans));
    System.exit(0);
}
//search in 3rd half ③7]
if(arr[rStart+1][cMid-1] >= target){
    int[] ans = BS(arr, ~~target~~ rStart+1, 0, cMid-1, target);
    System.out.println(Arrays.toString(ans));
    System.exit(0);
}
//search in 4th half [9 ④
if(arr[rStart+1][cMid-1] <= target){
        int BS(arr, rStart+1, cMid+1, cols-1, target);
        System.out.println(Arrays.toString(ans));
        System.exit(0);
}
}
```